

DAS-ILU: Distributed Incomplete LU Factorization

Performance Report

Dominik Gawel

June 2026

1 Algorithm Description

The implementation is based on the *DAS-ILU* algorithm. It computes a sparse $A \approx LU$ decomposition (ILU(0) sparsity constraint) distributed across P MPI processes, each owning rows $[kN/P, (k+1)N/P)$.

Row classification. Each locally-owned row is classified as *interior* (all lower-triangular non-zeros belong to the same rank) or *separator* (at least one lower-triangular non-zero belongs to a lower-numbered rank).

Symmetric permutation. Interior rows are reordered to come first in the local CSR block $(0, \dots, n_{\text{int}} - 1)$, separator rows last $(n_{\text{int}}, \dots, n_{\text{loc}} - 1)$. The same permutation is applied to both rows and columns. Figure 1 shows the resulting memory layout and the three-partition column encoding used in `lu_col`.

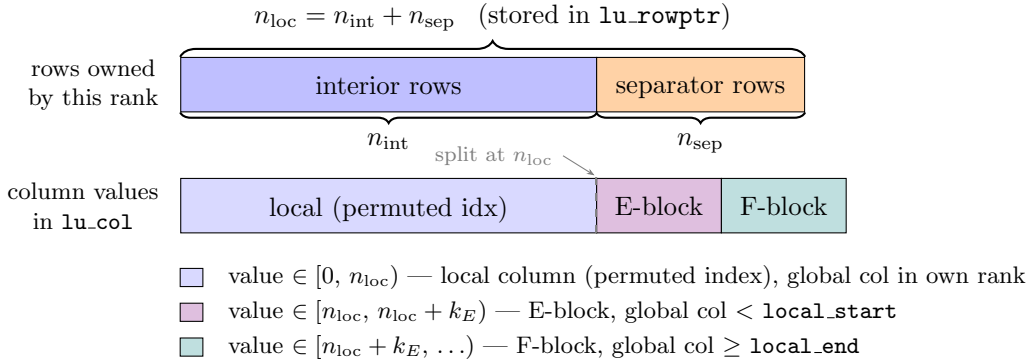


Figure 1: Local CSR memory layout after permutation. The column encoding uses n_{loc} as the boundary between local and external columns (E-block: owned by lower-ranked processes, F-block: owned by higher-ranked processes).

Factorization. The interior block is factorized once with standard row-oriented ILU(0), no communication required. The separator block is factorized iteratively: each iteration fetches the current U rows from lower-ranked processes via `MPI_Isend/MPI_Irecv`, then re-applies one ILU(0) sweep. Convergence criterion: $\|\Delta U_{\text{sep}}\|_F / \|U_{\text{sep}}\|_F < 10^{-8}$ checked via `MPI_Allreduce`, maximum 50 iterations. Because E-block pivots (globally earlier) must be eliminated before local lower-triangular pivots, the sweep uses a two-pass approach.

Triangular solve. Both $Ly = b$ and $Ux = y$ are solved with Gauss-Seidel iteration. Each iteration exchanges separator boundary values with neighboring ranks; convergence is declared when the update norm falls below 10^{-10} times the respective right-hand-side norm: $\|\Delta y\|_2 < 10^{-10} \|b\|_2$ for the forward sweep, $\|\Delta x\|_2 < 10^{-10} \|y\|_2$ for the backward sweep, up to 1000 iterations are allowed.

Differences from the Reference Paper

- **Triangular solve strategy.** The paper solves $Ly = b$ and $Ux = y$ with a *block Jacobi iterative scheme*: interior rows (no lower-triangular external dependencies) are solved exactly in the first iteration, and only separator rows require subsequent iterations using boundary values received from neighbouring processes. For backward substitution the paper additionally proposes resolving interior rows in one final parallel step *after* separator convergence, exploiting the claim that (under the paper’s row-classification) no separator row from a higher-ranked process can appear as a dependency of a local interior row in the upper triangle.

Our implementation differs in two respects. First, local updates use *Gauss-Seidel* order (newly computed values reused within the same sweep) rather than pure Jacobi, which typically halves the iteration count. Second, in backward substitution we apply the iterative process to *all* rows, including interior rows, rather than deferring them to a final step. This is necessary because under our row-classification criterion. A row is interior if and only if all its lower-triangular non-zeros belong to the same rank, an interior row of rank k can still depend on interior rows of rank $k+1$ via upper-triangular entries, so the paper’s backward-substitution optimisation is not directly applicable.

- **Separator pivot ordering.** The paper’s ILU(0) row kernel assumes column indices arrive in monotone global order. Our three-partition encoding places E-block columns at encoded positions $\geq n_{\text{loc}}$, which breaks monotonicity. We resolve this with a two-pass sweep: E-block pivots first (Pass 1), then local lower-triangular pivots (Pass 2). The paper does not address this detail.
- **Convergence thresholds.** Separator factorization uses a Frobenius-norm relative change threshold of 10^{-8} (max 50 iterations). The triangular solves use a relative L2 update-norm threshold of 10^{-10} (max 1000 iterations), normalised by the respective right-hand-side: $\|b\|_2$ for $Ly = b$ and $\|y\|_2$ for $Ux = y$. These were tuned to keep errors below the required 10^{-4} correctness bound.

2 Test Matrices

Ten matrices across four families are used (Table 1). Nine are generated synthetically, **ecology2** is a real-world matrix from the SuiteSparse Matrix Collection (McRae group), representing a 2D ecology diffusion model.

Table 1: Test matrix properties.

Family	Name	N	nnz	Structure
2D Laplacian	lap2d_100k	99 856	498 016	5-pt FD, 316×316 grid
	lap2d_400k	399 424	1 994 592	5-pt FD, 632×632 grid
	lap2d_1m	1 000 000	4 996 000	5-pt FD, 1000×1000 grid
3D Laplacian	lap3d_10k	10 648	71 632	7-pt FD, 22^3 grid
	lap3d_100k	97 336	668 656	7-pt FD, 46^3 grid
	lap3d_360k	357 911	2 475 131	7-pt FD, 71^3 grid
Banded	band_5k	5 000	104 890	half-bw $b = 10$, 21 nnz/row
	band_50k	50 000	5 047 450	half-bw $b = 50$, 101 nnz/row
	band_200k	200 000	20 197 450	half-bw $b = 50$, 101 nnz/row
Real-world	ecology2	999 999	4 995 991	2D diffusion, SuiteSparse

The 2D/3D Laplacians cover thin-to-wide bandwidth ratios relative to the local block height. Banded matrices are denser (≈ 101 nnz/row vs. ≈ 5 for 2D Laplacian), enabling cache-effect

super-linear speedup. The 3D Laplacian bandwidth grows as n^2 , so experiments are capped at $P \leq 64$.

3 Performance Evaluation

All runs were on **Okeanos** supercomputer: up to 16 nodes, 16 MPI processes per node (256 total), Cray MPI, CC -O3 -std=c++2a. Timings are from `MPI_Wtime`, solve time is the average of 20 calls.

3.1 Strong Scaling — Triangular Solve

Table 2: Strong-scaling solve times (seconds) and speedup.

P	lap2d_1m		lap3d_360k		band_200k		ecology2	
	T_s	S	T_s	S	T_s	S	T_s	S
1	0.0372	1.0×	0.01365	1.0×	0.08566	1.0×	0.03768	1.0×
2	0.0241	1.5×	0.00866	1.6×	0.05779	1.5×	0.02441	1.5×
4	0.0113	3.3×	0.00492	2.8×	0.02737	3.1×	0.01150	3.3×
8	0.00573	6.5×	0.00272	5.0×	0.00787	10.9×	0.00603	6.2×
16	0.00309	12×	0.00160	8.5×	0.00320	26.8×	0.00299	12.6×
32	0.00174	21×	0.00099	13.8×	0.00175	49.0×	0.00174	21.7×
64	0.000931	40×	0.00319*	4.3×	0.00106	80.6×	0.000917	41.1×
128	0.000581	64×	—	—	0.000568	151×	0.000594	63.4×
256	0.000355	105×	—	—	0.000349	245×	0.000361	104.4×

* bandwidth exceeds block height, many separator iterations needed.

2D Laplacian and ecology2 scale almost identically (both 5 nnz/row, 2D diffusion structure): 104–105× at $P = 256$, reaching 41% of ideal efficiency. The real-world irregular numbering of **ecology2** has no measurable impact on performance.

Banded matrices exhibit super-linear speedup (245× at $P = 256$, ideal 256×). At $P = 1$ the full ≈ 240 MB non-zero layout far exceeds CPU cache;=, at $P = 256$ each rank’s ≈ 940 KB fits in L3.

3D Laplacian scales well to $P = 32$ but degrades at $P = 64$ because its bandwidth (n^2) exceeds the local block height, turning most rows into separators that require many Gauss-Seidel iterations.

3.2 Strong Scaling — Factorization

Table 3: Factorization time (s) and separator iteration count.

P	lap2d_1m		band_200k		ecology2	
	T_f	It.	T_f	It.	T_f	It.
1	0.531	1	2.969	1	0.649	1
4	0.448	2	1.907	2	0.553	2
16	0.441	2	1.674	2	0.577	2
64	0.439	2	1.587	2	0.564	2
256	0.629	2	1.808	2	0.766	2

Factorization time is nearly flat across P for 2D matrices (separator convergence in 2 iterations), then rises at $P = 256$ due to `MPI_Allreduce` and communication-plan overhead. The banded matrix takes longer to factorize because of its higher nnz density.

3.3 Weak Scaling

2D Laplacian with $\approx 10,000$ rows per rank: solve time grows from 0.323 ms at $P = 1$ to 0.731 ms at $P = 256$ (a $2.26\times$ increase over $256\times$ more total work), demonstrating near-ideal weak scalability for the solve phase.

3.4 Correctness

All matrix/process-count combinations pass the factorization check ($|A_{ij} - (LU)_{ij}| < 10^{-4}$, sampled) and solve roundtrip check ($|b - LU \text{ solve}(b)| < 10^{-4}$, 8 vectors), with one exception: **band_5k** at $P = 256$ ($N/P \approx 19 < 2b = 20$). The local block is narrower than the bandwidth, so virtually all rows become separators and the Gauss-Seidel iteration cannot converge within the budget.

4 Code Optimisation

Two optimisations were implemented and benchmarked.

Opt 1 — cache diagonal pointers. Both `ILU_backward_sweep` and `ILU_multiply` originally located the diagonal entry of each CSR row by scanning from the row start on every call ($O(\text{nnz}/P)$ per solve). A `diag_ptr[i]` array is now computed once after factorization and reused. At $P = 1$ this yields **7–13%** solve-time reduction across all matrix families.

Opt 2 — pre-allocate MPI request vectors. The Gauss-Seidel loop was modified to declare `std::vector<MPI_Request>` outside the iteration rather than reconstructing it each step. *No measurable improvement* was observed ($\pm 5\%$ noise). The vectors hold at most 2 handles (≈ 16 bytes), so allocation cost is negligible compared to MPI latency. Consequently, this modification was discarded and is not included in the final implementation.

Table 4: Solve time (s): baseline vs. Opt1 vs. Opt2, selected matrix/P.

Matrix	P	Baseline		Opt1	Opt2
lap2d_1m	1	0.04078	0.03724 (− 8.7%)	0.03696 (−0.8%)	
band_50k	1	0.02200	0.01912 (− 13.1%)	0.01924 (+0.6%)	
band_200k	1	0.09525	0.08566 (− 10.1%)	0.08598 (+0.4%)	
lap2d_1m	256	0.000360	0.000355 (−1.4%)	0.000358 (+0.8%)	
band_200k	256	0.000352	0.000349 (−0.9%)	0.000334 (noise)	

5 Conclusion

DAS-ILU achieves good parallel efficiency for the triangular solve. At $P = 256$, `lap2d_1m` and `ecology2` reach $\approx 105\times$ speedup. Dense banded matrices exceed $245\times$ due to cache effects. Factorization scales poorly past $P \approx 8$ due to global convergence checks and communication overhead. Matrix bandwidth relative to local block height is the dominant structural factor: once bandwidth exceeds block height (3D Laplacian at high P) separator counts surge and performance degrades. Caching diagonal pointers gave a consistent 7–13% solve improvement at $P = 1$. MPI request pre-allocation had no effect, confirming that network latency is the solve bottleneck.